

Lecture # 12

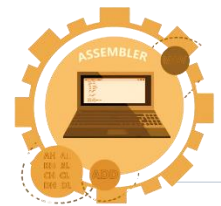
XLAT & XLATB, PTR Operator, INT 10H, INT 16H

Engr. Fiaz Khan



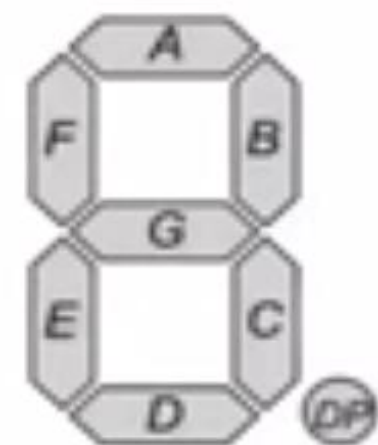
XLAT Instruction

- XLAT is used as translate data of AL register from Lookup table.
- First, You will have to make lookup table with the memory pointer BX.
- After that execute XLAT instruction
- $AL \leftarrow [BX+AL]$



Equivalent codes of seven Segment Display

0 1 2 3 4 5 6 7 8 9



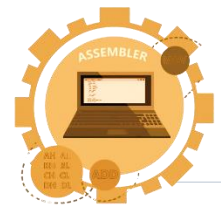
DP	G	F	E	D	C	B	A	DATA	INPUT
0	0	1	1	1	1	1	1	3FH	0'
0	0	0	0	0	1	1	0	06H	1'
0	1	0	1	1	0	1	1	5BH	2'
0	1	0	0	1	1	1	1	4FH	3'
0	1	1	0	0	1	1	0	66H	4'
0	1	1	0	1	1	0	1	6DH	5'
0	1	1	1	1	1	0	1	7DH	6'
0	0	0	0	0	1	1	1	07H	7'
0	1	1	1	1	1	1	1	7FH	8'
0	1	1	0	1	1	1	1	6FH	9'

XLAT Instruction – Example

```
MOV BX, 2050H  
MOV AL, 05H  
XLAT
```

Here 0', 1', 2', 3', 4', 5'... are the equivalent codes of 0, 1, 2, 3, 4, 5... respectively.

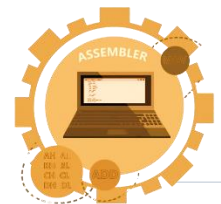
	Address	Data
	204FH	
BX	2050H	0'
	2051H	1'
	2052H	2'
	2053H	3'
	2054H	4'
BX+AL	2055H	5'



Little Endian Order

- Little endian is a way of storing multi-byte data types (such as integers) in memory, where the least significant byte (LSB) is stored at the lowest memory address and the most significant byte (MSB) is stored at the highest memory address.
- For example, consider the 32-bit hexadecimal value 0x12345678. In little endian order, it would be stored as follows:

Address	Data
-----	-----
Start	78h ; LSB
Start+1	56h
Start+2	34h
Start+3	12h ; MSB

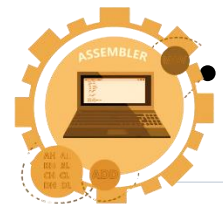


PTR Operator

- In assembly language, the PTR operator is used in conjunction with data declarations to specify the size of the data being defined. It's commonly used in NASM (Netwide Assembler) syntax.

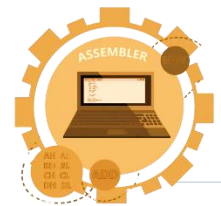
name PTR type

- name: The name of the variable or label being declared.
- PTR: Stands for "pointer".
- type: Specifies the size and type of the data. It can be one of the following:
- BYTE: 8-bit data (1 byte)
- WORD: 16-bit data (2 bytes)
- DWORD: 32-bit data (4 bytes)
- QWORD: 64-bit data (8 bytes)



PTR Operator

- To declare an array of bytes named myData with values 12h, 34h, 56h, and 78h:



Example

```
.model small
.stack 100h

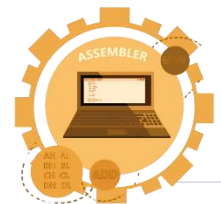
.data
    myData DB 12h, 34h, 56h, 78h

.code
main PROC
    mov ax, @data
    mov ds, ax

    mov ax, word PTR [myData]    ; AX = 3412h
    mov ax, word PTR [myData+2] ; AX = 7856h
    mov eax, DWORD PTR myData    ; EAX= 78563412h

    mov ah, 4Ch                ; Exit program
    int 21h
main ENDP

END main
```



Over to You Guys.

.data

varB BYTE 65h,31h, 02h, 05h

varW WORD 6543h,1202h

varD DWORD 12345678h

.code

mov ax, WORD PTR [VARB+2] ;

mov bl, BYTE PTR VARD ;

mov bl, BYTE PTR [varW+2] ;

mov ax, WORD PTR [varD+2] ;

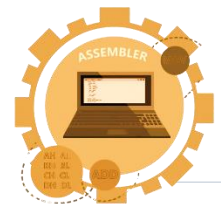
mov eax, DWORD PTR varW ;

mov ah, 4Ch ; Exit program

int 21h

main ENDP

END main



Result Match?

.data

varB BYTE 65h,31h, 02h, 05h

varW WORD 6543h,1202h

varD DWORD 12345678h

.code

mov ax, WORD PTR [VARB+2] ; 0502h

mov bl, BYTE PTR VARD ; 78h

mov bl, BYTE PTR [varW+2] ; 02h

mov ax, WORD PTR [varD+2] ; 1234h

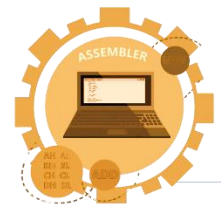
mov eax, DWORD PTR varW ; 12026543h

mov ah, 4Ch ; Exit program

int 21h

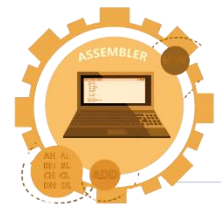
main ENDP

END main



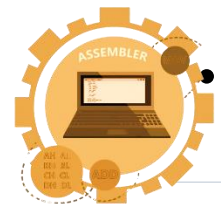
INT 10H

- In assembly language, int 10h is a BIOS interrupt used to perform various video services.
- It allows direct interaction with the display hardware, such as setting video modes, changing cursor position, displaying characters, and more, depending on the specific function called.



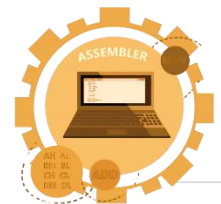
INT 10H –Service Routines

- Set Video Mode (AH = 0): Sets the video display mode.
- Set Cursor Position (AH = 2): Sets the cursor position on the screen.
- Get Cursor Position and Size (AH = 3): Retrieves the current cursor position and size.
- Read Character and Attribute at Cursor (AH = 8): Reads the character and attribute at the current cursor position.
- Write Character and Attribute at Cursor (AH = 9): Writes the character and attribute at the current cursor position.
- Scroll Window Up (AH = 6): Scrolls the entire window up by one or more lines.
- Scroll Window Down (AH = 7): Scrolls the entire window down by one or more lines.
- Get Video Mode (AH = 0Fh): Retrieves the current video mode.



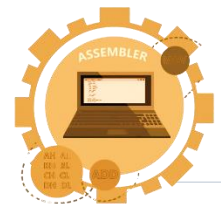
Program to draw the box

```
;program to draw the box
.model small
.stack 100h
.data
.code
main PROC
    mov ah,6
    mov al, 10
    mov bh,01010000b
    mov ch,0
    mov cl,0
    mov dh, 50
    mov dl, 50
    int 10h
    mov ah, 4Ch    ; Exit program
    int 21h
main ENDP
END main
```



INT 16H -Introduction

- Assembly Language Interrupt: int 16h
- Purpose: Keyboard Input
- Function: Triggers a software interrupt for BIOS keyboard services



INT 16H -Introduction

```
mov ah, 0      ; Wait for a keypress
```

```
int 16h        ; Call interrupt 16h
```

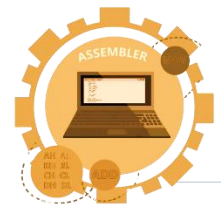
```
mov dl, al     ; Move ASCII code to DL
```

```
mov ah, 2      ; Print character function
```

```
int 21h        ; Call interrupt 21h to display character
```

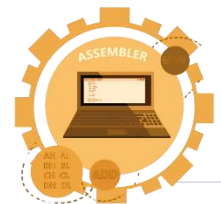
```
mov ah, 4Ch    ; Terminate program
```

```
int 21h        ; Call interrupt 21h to exit
```



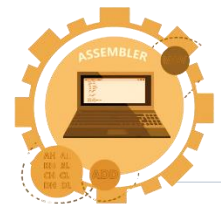
Explanation

- `mov ah, 0`: Set AH register for keyboard input
- `int 16h`: Call BIOS keyboard service interrupt
- `mov dl, al`: Move ASCII code to DL register
- `mov ah, 2`: Set AH for character output
- `int 21h`: Call BIOS character output interrupt
- `mov ah, 4Ch`: Set AH for program termination
- `int 21h`: Call DOS program termination interrupt



Conclusion

- int 16h is a key instruction for keyboard input in assembly.
- It utilizes BIOS services for keyboard handling.
- Combining with other interrupts like int 21h, it enables interactive assembly programs.





THANK YOU!

Do you have any questions?

ENGR. MUHAMMAD FIAZ

LECTURER COMPUTER SCIENCE

TELF/EF SET / ACEPT CERTIFIED

MICROSOFT ACADEMY TRAINER

ORACLE CERTIFIED PROFESSIONAL

GOOGLE CERTIFIED PROFESSIONAL

Lahore, Pakistan

+92-320-7617-093

muhammad.fiaz@cs.uol.edu.pk

<https://www.linkedin/in/fiazofficials>

